CS-107 Lecture 6
Data Representation

Troy Astarte

Megan Venn-Wycherley

Recap: binary addition and subtraction

- **Addition:** work right to left. 1 + 1 = 10, so write 0 and carry 1 into the next column.
- **A carry off the top adds a bit:** 10110110 + 01101101 = 100100011, that is 182 + 109 = 291.
- **Subtraction:** borrow from the column to the left, and the borrow cascades through any zeros.
- **Leading zeros carry no value,** so 1000000 - 0000001 = 111111, that is 64 - 1 = 63.

Recap: binary multiplication

- **The method:** multiply by each digit of the second number, shifting one place left each time, then add the partial products.
- **Binary makes it easy:** a 1 contributes a shifted copy of the first number, a 0 contributes nothing.
- **Worked example:** 1011 x 101 = 1011 + 101100 = 110111, that is 11 x 5 = 55.
- **Shifting left one place doubles a number,** which is why multiplication in hardware is shift and add.

Recap: representing negative numbers

- **n bits represent 2^n things,** so any scheme has to divide that fixed space between positive and negative values.
- **Sign-magnitude:** first bit for the sign, the rest for the magnitude. It gives two zeros and awkward arithmetic.
- **Two's complement:** the first bit still marks negative, but the values wrap around, so there is one zero and one rule for adding.
- **Negate by inverting every bit and adding 1:** 0011 is 3, so 1100 + 1 = 1101 is -3.

Recap: two's complement and overflow

- **Addition is unchanged:** add the bit patterns and throw away any carry beyond the top bit.
- **Subtraction needs no new method:** a - b = a + (-b), so 0010 + 1101 = 1111, that is 2 - 3 = -1.
- **Each width fixes a range:** 8 bits hold 0 to 255 unsigned, or -128 to 127 signed; 16 bits reach 32767.
- **Leave the range in either direction and the value overflows,** which is how Ariane 5 was lost.

Goals for Today

- Representing real numbers in binary
- Floating point and IEEE 754
- Encoding text: ASCII and Unicode
- Colour, images and sound
- Trading detail against size

Finite Data in an Infinite World

Real World Data

How do we begin to measure the infinite data around us?

- **The world is continuous:** between any two shades of red, or any two moments in time, there are infinitely many values.
- **A machine is finite:** n bits give 2^n patterns, so 8 bits hold 256 values and 24-bit colour holds 16.7 million.
- **So every representation is a choice:** how far the values go (range) and how finely we divide them (precision).
- **The test is fitness for purpose:** too little detail and the answer is wrong, too much and we waste storage and time.

A CD keeps 44,100 samples a second at 16 bits each. It is not the real waveform, but no listener can tell.

How do we begin to represent data?

- All data on a computer system is represented using binary patterns, which are sequences of 1s and 0s.

- A bit is the fundamental unit of information - we can't store or use anything smaller than that - but we often deal with larger collections of bits.

Representing Real Numbers

Real Numbers in decimal

- A number with a whole part and a fractional part (either of which may be zero):
  - 17.0
  - 3.333333…
  - 1.41421356…
  - 0.002

- Note that fractional part may be **infinite**.
- In decimal, positions to the right of the point are tenths, hundredths, thousandths, etc.: 10^{-1}, 10^{-2}, 10^{-3}…

Real Numbers in binary

- The same happens in binary.
- Positions to the right of the **radix point** are:
  - Halves (2^{-1})
  - Quarters (2^{-2})
  - Eighths (2^{-3})
  - …

The name of the "point" is actually the 'Radix Point'.

We tend to call it a "decimal point", but that's because we often work in decimal.

…	2^2	2^1	2^0	.	2^{-1}	2^{-2}	…
…	Fours	Twos	Ones	.	Halves	Quarters	…

Real Numbers in binary

- To convert the value we can use the calculations we have seen before, taking into account the new negative positions in the fractional portion of the number.

- Example:

10.0101_2 =

2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}
1	0	.	0	1	0	1

= 2 + 0 + 0 + 0.25 + 0 + 0.0625

= 2.3125_{10}

Floating Point representation

- Now we have a complicated number (whole and fractional part)
- A Real Number can be defined by the formula:

$$\text{sign} \cdot \text{mantissa} \cdot \text{base}^{\text{exponent}}$$

- This representation is called **floating point**, because the radix point "floats".
- We will assume that both the mantissa and the exponent fits within some pre-agreed number of bits.

Floating Point representation

$$\text{sign} \cdot \text{mantissa} \cdot \text{base}^{\text{exponent}}$$

Sign: +1 or -1, represented as 0 and 1, respectively. (think back to Two's Complement). This is a single bit.

Mantissa: Significant digits of the number being represented. This is an integer.

Base: Base used. Often pre-determined (binary), so no need to store.

Exponent: Scales our number by shifting the position of the floating radix point. This is an integer.

Floating Point example

To represent π = 3.14159265358979... with a finite number of bits:

Round the number to some number of significant digits, say 3.14159

Rewrite as floating point:

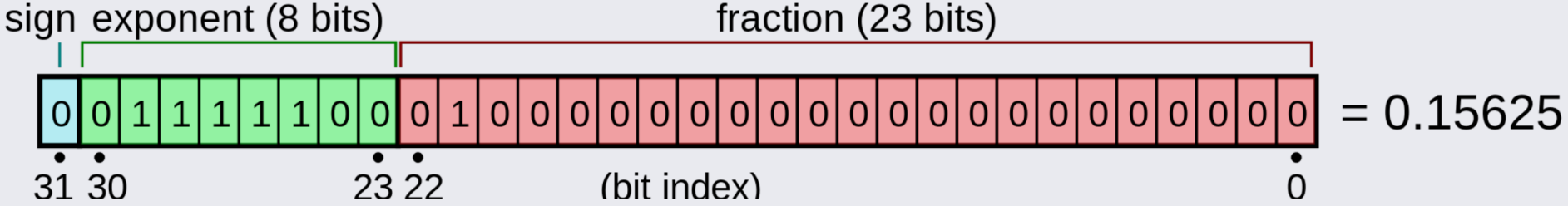
$$+1 \cdot 314159 \cdot 10{-}5$$

Represent +1, 314159 and -5 as integers, as in previous slides

IEEE 754 Standard for Floating Point

Float

Single precision floating point.

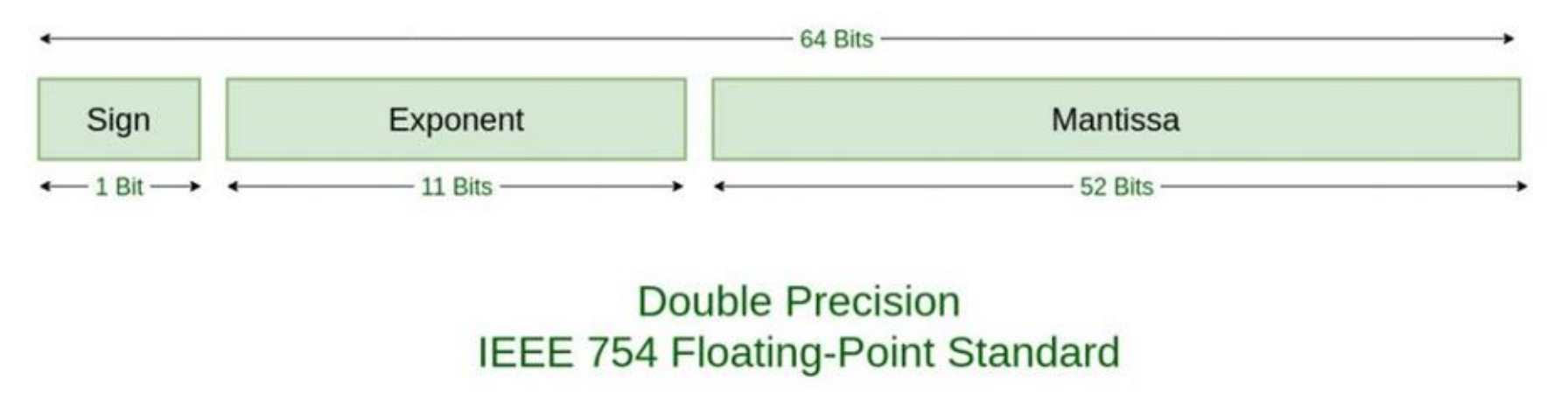


Number encoded is

$$(-1)^{b_{31}} \cdot (1.b_{22}b_{21}\ldots b_0)_2 \cdot 2^{(b_{30}b_{29}\ldots b_{23})_2 - 127}$$

IEEE 754 Standard for Floating Point

- There are other standards on number representation. For example the following gives higher precision floating point values than the Single Precision on the last slide:



https://www.geeksforgeeks.org/ieee-standard-754-floating-point-numbers/

Base conversion with real numbers

- Remember our method for calculating an integer number with positional notation:

While (value is not zero)
Divide value by the new base
Store remainder
Replace value with the quotient
Read remainders in reverse order

- A similar method holds for the fraction component in a real number...

Base conversion with real numbers

- To calculate the value of the fractional part, we now want to scale up (due to the negative exponent):

While (value is not zero OR precision is reached)
    Multiply value by the new base
    Store the whole part
    Replace value with the fractional part of the result
Read whole parts in order

Example: Convert 3.625_{10} to binary:

Whole part (3), use **division** and **remainder** method:

3 / 2 = 1, remainder: 1
1 / 2 = 0, remainder: 1

Read remainders **in reverse**:
11

Fractional part (0.625), use **multiply** and **whole** method:

0.625 * 2 = 1, fraction: 0.25
0.25 * 2 = 0, fraction: 0.5
0.5 * 2 = 1, fraction: 0

Read wholes **in order**: 101

Example: Convert 0.3_{10} to binary:

Whole part (0):

0 / 2 = 0, remainder: 0

Read remainders **in reverse**: 0

Fractional part (0.3):

0.3 * 2 = 0, fraction: 0.6
0.6 * 2 = 1, fraction: 0.2
0.2 * 2 = 0, fraction: 0.4
0.4 * 2 = 0, fraction: 0.8
0.8 * 2 = 1, fraction: 0.6
0.6 * 2 = 1, fraction: 0.2
…



Read wholes **in order**: 010011…

Scientific notation

A form of floating-point representation in which the decimal point is kept to the right of the leftmost digit

12001.32708 becomes 1.200132708E+4 in scientific notation (E+4 is how computers display $x10^4$)

What is 123.332 in scientific notation?

What is 0.0034 in scientific notation?

Representing Text

Representing Text

- Simple Idea:

  Assign a binary bit pattern per character you want to represent. Text is then stored as a sequence of bit patterns.

- Not so simple implementation:
  - There are many characters (different alphabets, mathematical symbols, emojis etc.)
  - Some characters are modifications of others: e, è, é, ë
  - Direction of the character sequence can matter: Arabic, Hebrew etc.

Character set standards

ASCII (1960's) 7-bit encoding.

EBCDIC (1960's) 8-bit encoding.

ISO 8859 (1987) 8-bit encodings.

- ISO 8859-1 also known as Latin-1.

Unicode (1993–) 143,859 characters covering 154 modern and historic scripts.

- UTF-8
- UTF-16
- UTF-32

ASCII

- Originally 7 bits, providing 128 unique characters.
- Later extended to use 8 bits. Now each character is a byte.
- First 32 characters aren't normal characters, what do they do?
- ASCII doesn't cover advanced international requirements

Left Digit(s) \ Right Digit	ASCII 0	1	2	3	4	5	6	7	8	9
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT
1	LF	VT	FF	CR	SO	SI	DLE	DC1	DC2	DC3
2	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS
3	RS	US	□	!	"	#	$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	DEL		

Unicode

- More bits used:
  - Commonly: 16 bits
  - Sometimes: 8 or 32 bits
- First 256 correspond to the extended ASCII set
- UTF-16 allows 65536 different characters



https://en.wikipedia.org/wiki/Emoji#Unicode_blocks

Some Unicode examples

Character Symbol	Hexadecimal value	Name
Ä	C4	Latin Capital Letter a with Diaeresis
Д	414	Cyrillic Capital Letter De
ऄ	906	Devanagari Letter Aa
襄	342E	Ideograph: to help; to assist, to achieve, to rise; to raise
	1300A	Egyptian Hieroglyph A008; rejoice, celebrate, to be jubilant
	1F601	Grinning Face with Smiling Eyes Emoji

Representing Colours

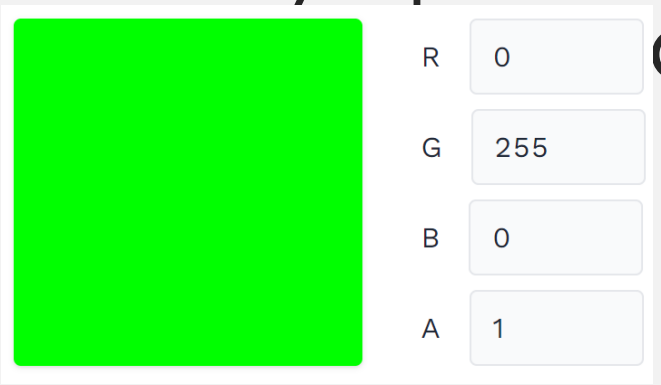
Representing colours

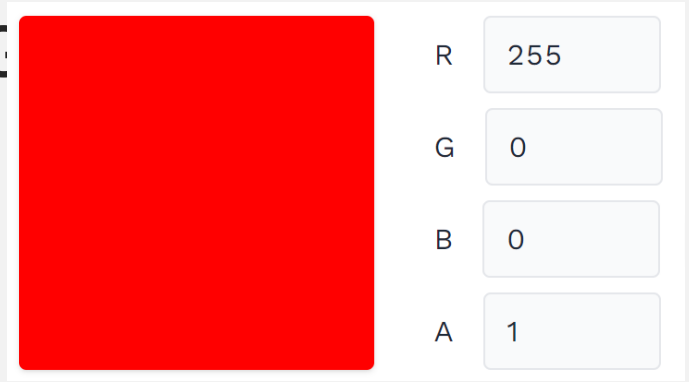
- What is colour? Essentially it is an attribute caused by things reflecting or emitting certain wavelengths of light.

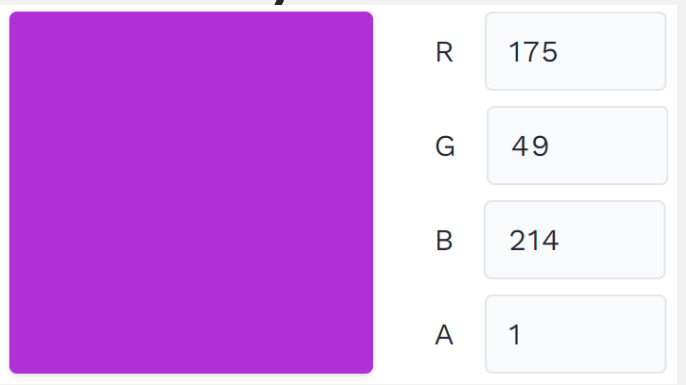
- We perceive colour with **photoreceptors** in our eyes, these cells respond to particular wavelengths and our brain interprets this as a colour.

Representing colours

- One way represent a colour is to encode the intensity of **red**, G







Give it a try [RGBA Color Picker - https://rgbacolorpicker.com/](https://rgbacolorpicker.com/)

RGB colours

- Assuming that we use 8 bits per channel, then we can encode colours as triples of numbers (red, green, blue) in the range 0-255.
  - 0 represents no presence, 255 represents maximum presence.

- Examples:
  - (0,255,0) is full green, but no red or blue. This looks green.
  - (255,255,255) is full red, green and blue. This looks white .
  - (150,75,0) is some red, less green, and no blue. This looks brown.

- Hexadecimal is often used for colours, for example the brown colour above can be written as #964B00.

Other colour encodings

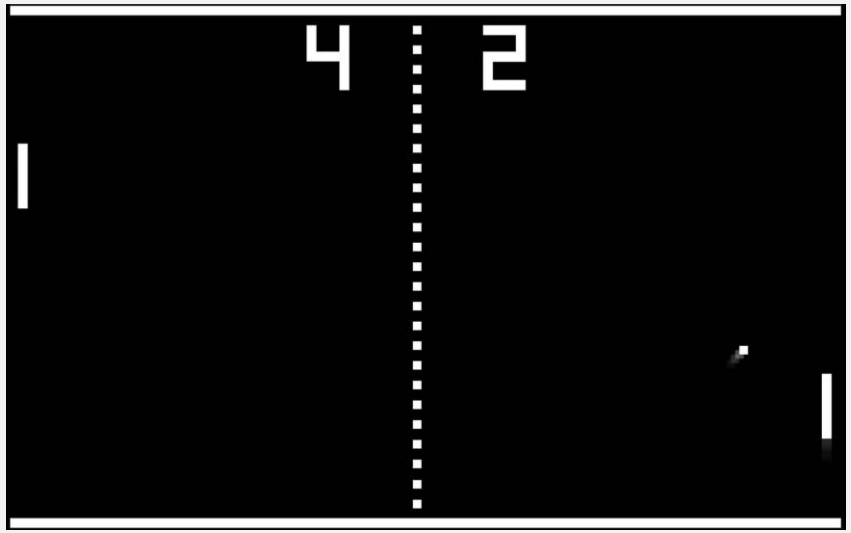
- Apart from RGB encoded colours there are many other schemes:
  - **HSV** (hue, saturation, value) and related models.
  - **CMY** (cyan, magenta, yellow).
  - **CMYK** (cyan, magenta, yellow, black). Popular in printing process.

- **Colour depth**
  - Number of bits used to encode colour can vary. 8 bits is standard, but some situations use more bits. More bits, more colours representable.
  - Humans can perceive ~10 million different colours, and 24 bits would give us the ability to represent over 16.7 million.

Representing Images

Representing Images

- There's a number of ways in which we can represent images.





Digitized images and graphics

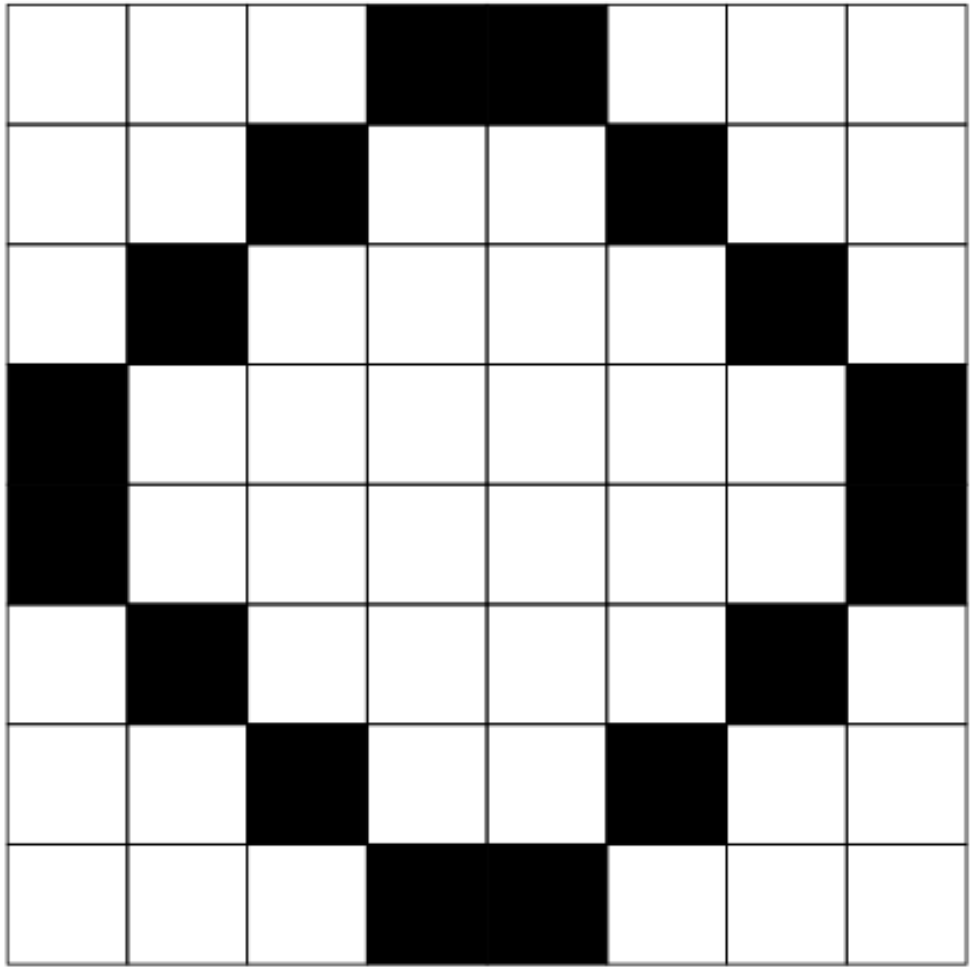
- Need to represent shapes and colours within an image.
- Requires discretization of the observation in 2D space, and in colour.
- **Pixels**
  - Dots of colour in image (or display device)
- **Resolution**
  - Number of pixels in image (or device)

Vector vs Raster Graphics

- **Raster Graphics**
  - Treat image as collection of pixels, encoding the colour for each.
  - Can be larger in size. Resizing leads to pixelation.
  - Most common formats: BMP, GIF, PNG, and JPEG

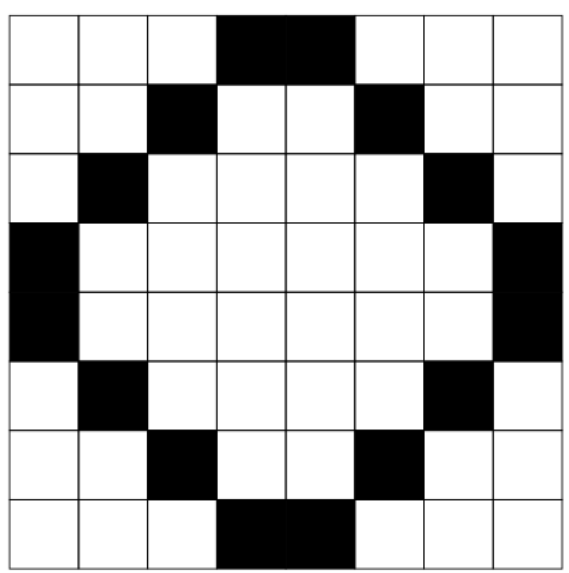
- **Vector Graphics**
  - Treat image as collection of mathematically defined geometric objects, in the image. Describing direction, length, thickness.
  - Re-sizing is easy, little pixelation.
  - Common formats: SVG

How could this image be represented?

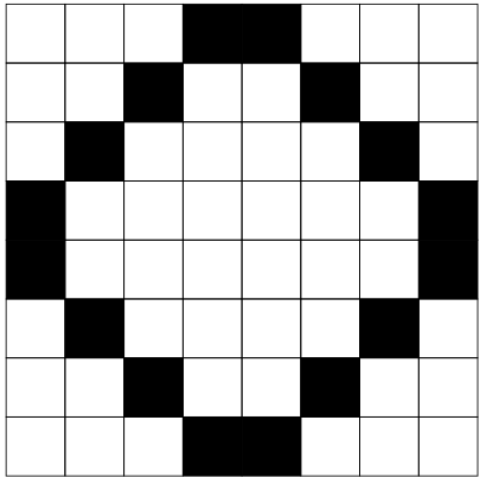


Binary Images

- Split them up into grids of pixels (picture elements), and represent the pixels numerically, e.g. as an intensity value.
- E.g.



```
0 0 0 1 1 0 0 0
0 0 1 0 0 1 0 0
0 1 0 0 0 0 1 0
1 0 0 0 0 0 0 1
1 0 0 0 0 0 0 1
0 1 0 0 0 0 1 0
0 0 1 0 0 1 0 0
0 0 0 1 1 0 0 0
```

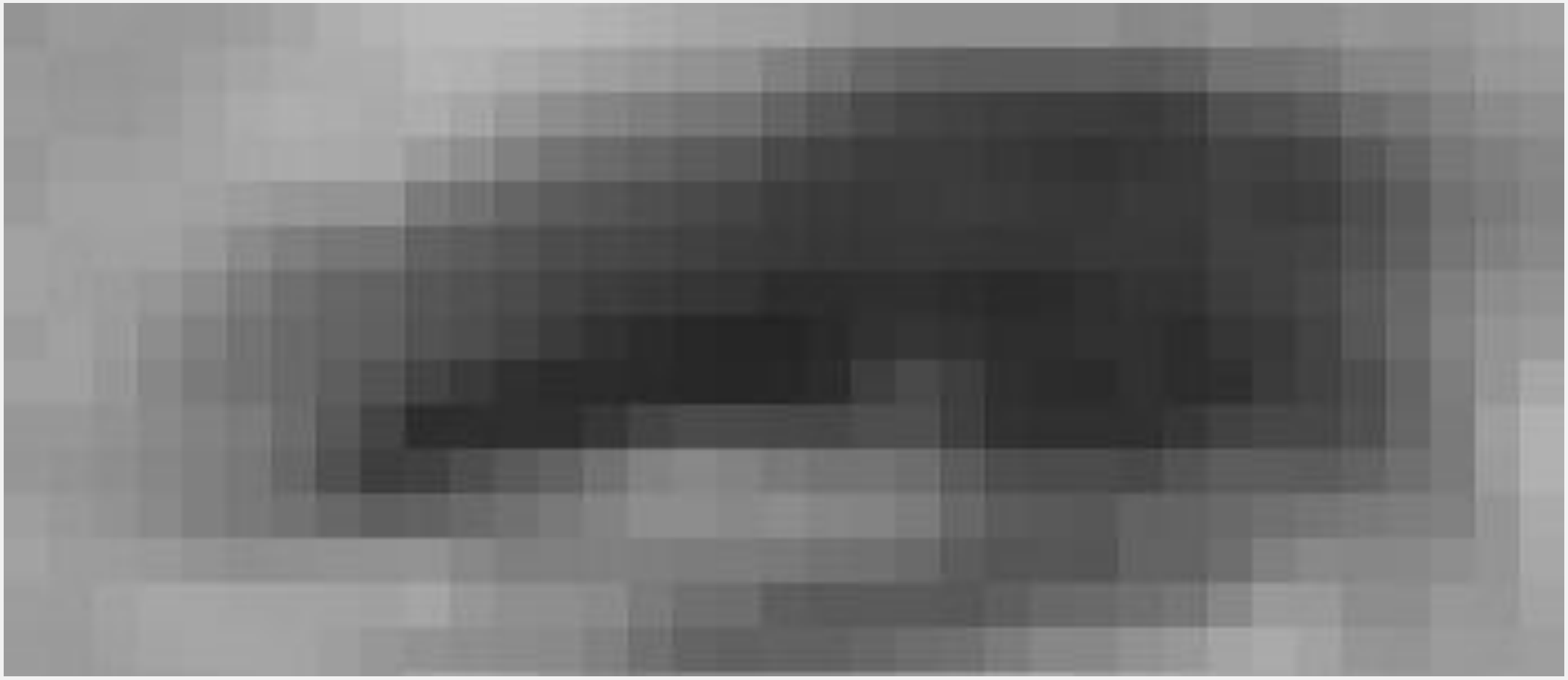


0 0 0 1 1 0 0 0
0 0 1 0 0 1 0 0
0 1 0 0 0 0 1 0
1 0 0 0 0 0 0 1
1 0 0 0 0 0 0 1
0 1 0 0 0 0 1 0
0 0 1 0 0 1 0 0
0 0 0 1 1 0 0 0

Binary Images

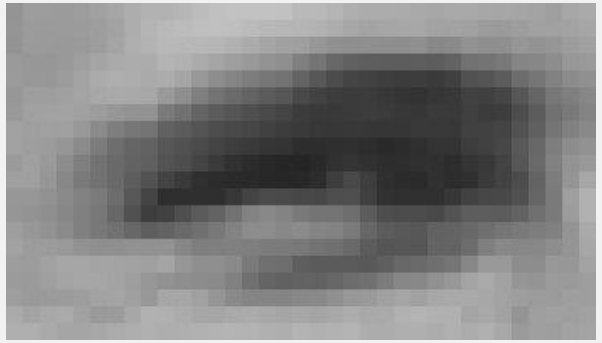
- Each pixel is just "on or off", so we can encode it with 1 bit per pixel; each row is thus given by the binary numbers beside it, whose decimal values are listed alongside. This shows how to represent a simple 8 x 8 1-bit image as 8 bytes (i.e. 64 bits), in a very simple and natural manner.

How could this image be represented?



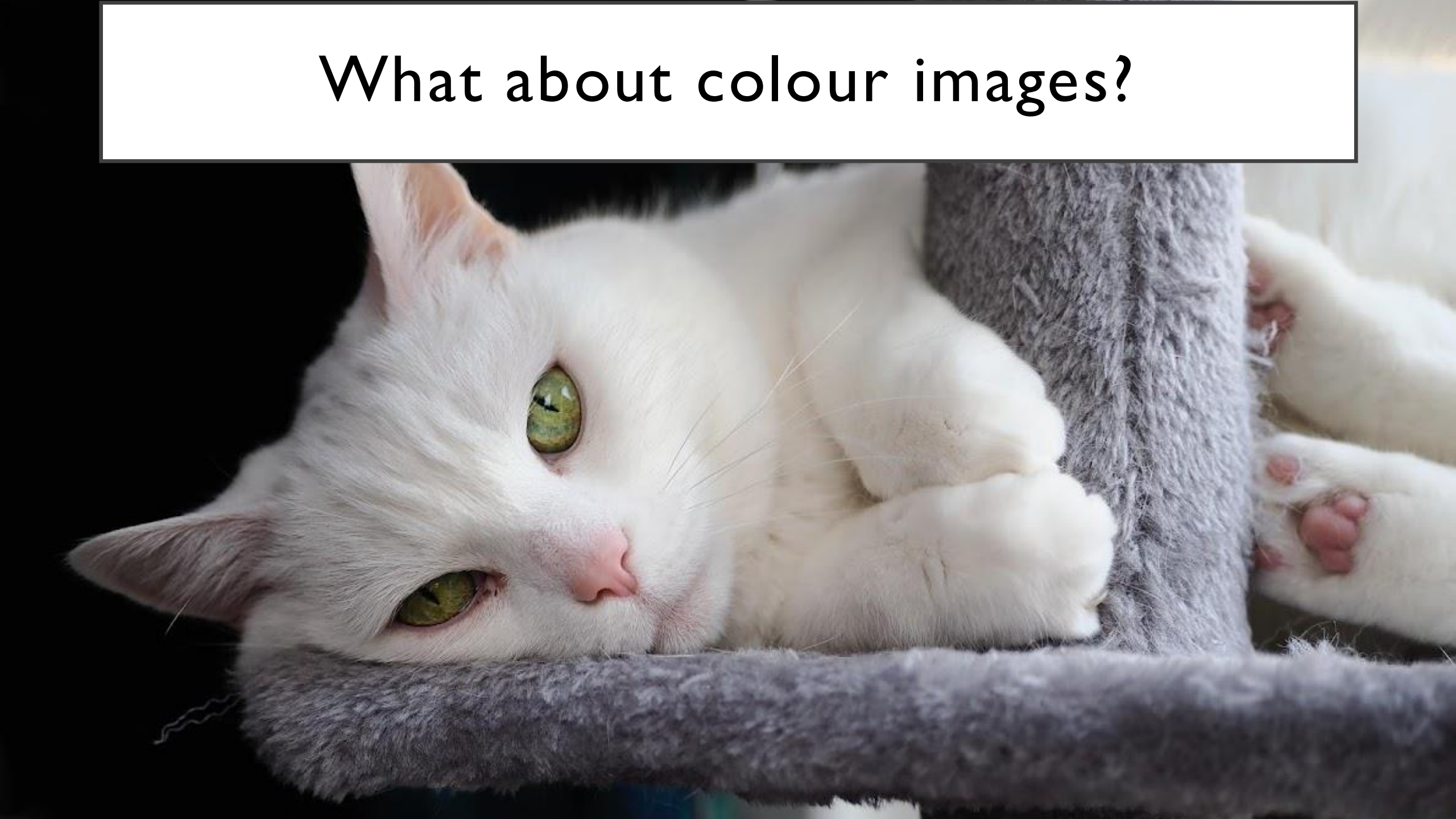
Greyscale Images

- We could use one byte per pixel, representing amount of light, where 0 is black, 255 is white, and all other value are varying degrees of grey (a greyscale image).



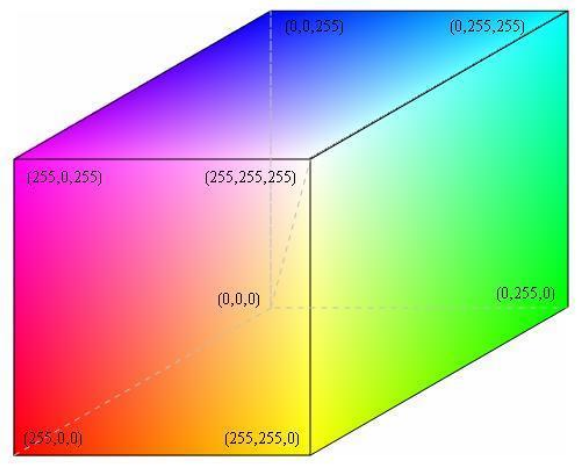
- 00000000 = Black
- 11111111 = White
- All other values are greys!

What about colour images?



Images

- For colour, we could use 3 bytes per pixel, containing values for RED, GREEN, and BLUE:



0	0	0	black
255	255	255	white
127	127	127	mid grey
255	0	0	bright red
212	17	184	brightish purpley
			etc.

Size of images?

- An 800x600 image has 480,000 pixels. If it's in 24 bit colour, that's 3 bytes per pixel, so a total of 1,440,000 bytes - about 1.37MB. A 1600x1200 image would be four times bigger: five and a half megabytes.

- But I have such an image on my hard drive and it is only 32,207 bytes big: one 44th of the size! It's a JPEG image, utilising some sophisticated compression techniques.

JPEG Images

(Joint Photographic Experts Group)

- JPEG is actually a compression scheme, looking to approximate the pixel-wise colours within the image.
- It averages colour hues over short distances, lowering the need to store every single pixel value.
- **Why:** Human vision tends to blur colours together within small areas, so no need to be perfect.
- **How:** Transform from the spatial domain to the frequency domain, then discard high frequency component. Argh, maths!

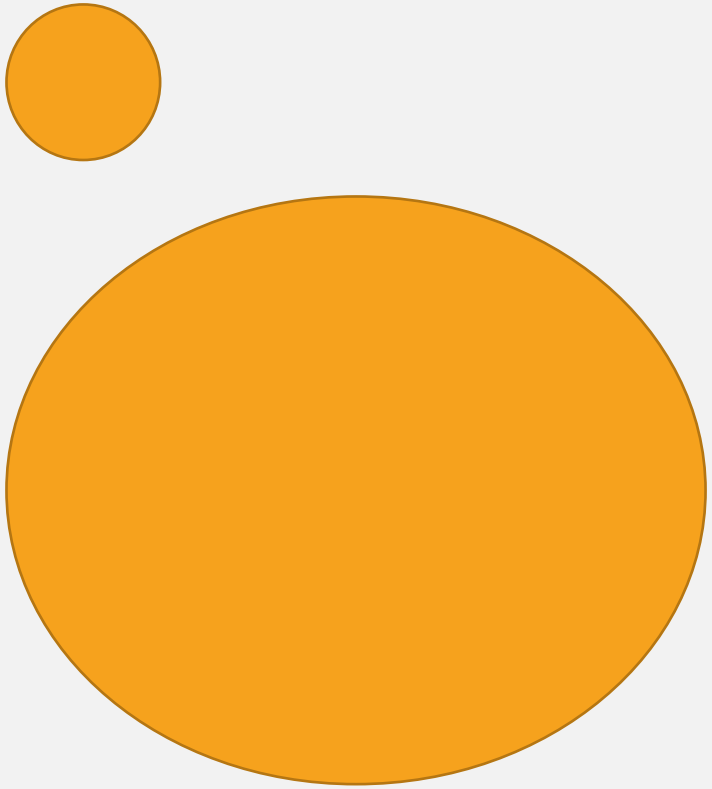
JPEG pixelation





SVG

- As the circles are mathematically defined, we can make them larger/smaller and pixelation does not occur.
- Sometimes the representation of a vector graphic is smaller in size compared to a raster version, as not every pixel is stored.
- However, requiring a mathematical definition for the objects means that it doesn't work well for representing natural photographs.



Representing Sound

Representing Sound

- Sound is the perception of pressure changes within the air (roughly), so convert pressure to digital signal with membrane (speaker / mic)

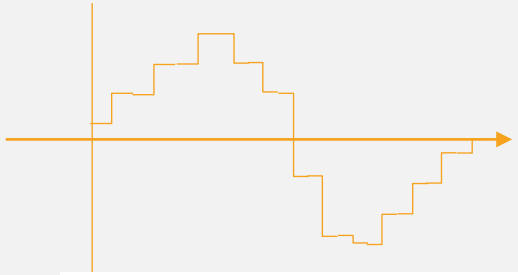
- Record air pressure on the membrane, sampled at periodic intervals. This is the **sampling frequency**



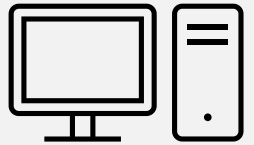


Analog Signal





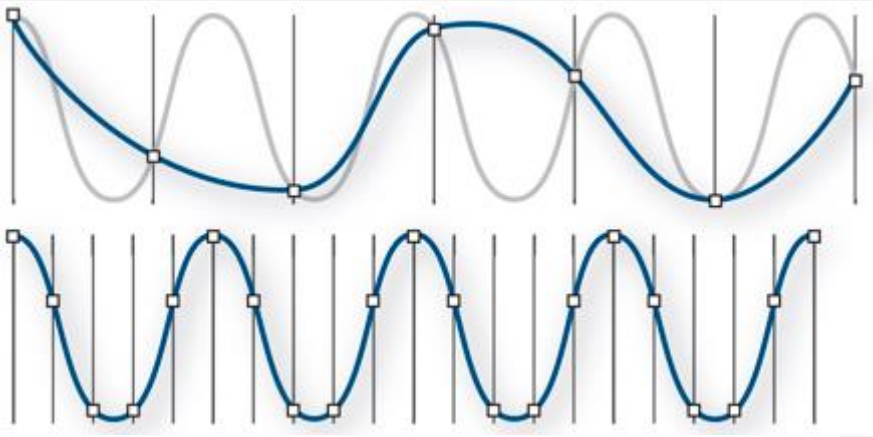
Digital Signal



Representing Sound

- The frequency at which we sample the air pressure allows us to more faithfully represent the original continuous frequency - Humans can hear up to a frequency of 20 kHz

- We often sample at higher frequencies:
  - ~11kHz – AM Radio
  - ~44kHz – CD quality
  - ~48kHz – DVD quality
  - 96kHz – Blu-ray quality



Digitizing audio in Audition (adobe.com)



Better sound?

- Sample more frequently (44100Hz for ‘CD quality’)
- Use more bytes per sample: e.g. with 2 bytes per sample, each sample is in the range 0 to 2^{16} - 1, e.g. 0 to 65535 (‘16 bit quality’)

Sound

- 16-bit CD-quality stereo sound is 2 bytes per sample, 44,100 samples per second, across 2 channels.
  - That's 2 x 44100 x 2 = 176,400 bytes per second, or about 10 MB per minute.

- MP3 audio, like JPEG for images, utilises (different) data compression to allow us to actually store music at a rate of about 1 MB per minute - around a tenth of the 'normal' uncompressed size.

Why is this important?

Goals for Today

- Representing real numbers in binary
- Floating point and IEEE 754
- Encoding text: ASCII and Unicode
- Colour, images and sound
- Trading detail against size

CS-107 Lecture 6
Data Representation

Troy Astarte

Megan Venn-Wycherley